# Network Dyadic Models

A common challenge in network analysis is structural dependency, particularly when events are unevenly distributed and exhibit zero-inflation, partial pooling across dyads, or linked sender/receiver variance. This chapter covers three advanced model structures mapped from their original Stan formulations to their equivalent BI syntax in **Python**, **R**, and **Julia**.

---

## 1. Binomial Edge Weight Model

This model assesses event probability when events occur as a proportion of total duration or valid observation periods (the "divisor"). It incorporates *optional zero-inflation* and *partial pooling of edge weights*.

### Python

```
@BI
def model_binomial(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                   event, divisor, dyad_ids, design_fixed, design_random, random_group_index
                   prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                   prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                   prior_zero_prob_alpha, prior_zero_prob_beta,
                   priors_only, partial_pooling, zero_inflated):

    # Edge weights
    if partial_pooling == 0:
        edge_weight = m.dist.normal(prior_edge_mu, prior_edge_sigma, shape=(num_edges,))
    else:
        edge_sigma = m.dist.normal(0, prior_edge_sigma, shape=(1,), lower=0)
        edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[0], shape=(num_edges,))
```

```
    # Fixed effects
    beta_fixed = 0
    if num_fixed > 0:
        beta_fixed = m.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=(num_fixed,))

    # Random effects
    beta_random = 0
    if num_random > 0:
        random_group_mu = m.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape=
        random_group_sigma = m.dist.normal(0, prior_random_std_sigma, shape=(num_random_group
        beta_random = m.dist.normal(random_group_mu[random_group_index], random_group_sigma[

    # Predictor
    predictor = jnp.zeros(num_rows)
    if num_edges > 0:
        predictor += edge_weight[dyad_ids]
    if num_fixed > 0:
        predictor += jnp.dot(design_fixed, beta_fixed)
    if num_random > 0:
        predictor += jnp.dot(design_random, beta_random)

    if not priors_only:
        p = m.link.inv_logit(predictor)
        if zero_inflated == 0:
            m.dist.binomial(total_count=divisor, probs=p, obs=event)
        else:
            zero_prob = m.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=(1,))
            m.dist.zero_inflated_binomial(gate=zero_prob[0], total_count=divisor, probs=p, ob
```

**R**

```
model_binomial <- function(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                   event, divisor, dyad_ids, design_fixed, design_random, random_group_index
                   prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                   prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                   prior_zero_prob_alpha, prior_zero_prob_beta,
                   priors_only, partial_pooling, zero_inflated) {

    # Edge weights
    if (partial_pooling == 0) {
        edge_weight = bi.dist.normal(prior_edge_mu, prior_edge_sigma, shape=c(num_edges))
```

```
    } else {
        edge_sigma = bi.dist.normal(0, prior_edge_sigma, shape=c(1), lower=0)
        edge_weight = bi.dist.normal(prior_edge_mu, edge_sigma[1], shape=c(num_edges))
    }

    # Fixed effects
    beta_fixed = 0
    if (num_fixed > 0) {
        beta_fixed = bi.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=c(num_fixed))
    }

    # Random effects
    beta_random = 0
    if (num_random > 0) {
        random_group_mu = bi.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape
        random_group_sigma = bi.dist.normal(0, prior_random_std_sigma, shape=c(num_random_gr
        beta_random = bi.dist.normal(random_group_mu[random_group_index], random_group_sigma
    }

    # Predictor
    predictor = jnp$zeros(num_rows)
    if (num_edges > 0) {
        predictor = predictor + as.vector(edge_weight[dyad_ids])
    }
    if (num_fixed > 0) {
        predictor = predictor + jnp$dot(design_fixed, beta_fixed)
    }
    if (num_random > 0) {
        predictor = predictor + jnp$dot(design_random, beta_random)
    }

    if (!priors_only) {
        p = m$link$inv_logit(predictor)
        if (zero_inflated == 0) {
            bi.dist.binomial(total_count=divisor, probs=p, obs=event)
        } else {
            zero_prob = bi.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=c(1)
            bi.dist.zero_inflated_binomial(gate=zero_prob[1], total_count=divisor, probs=p, o
        }
    }
}
```

### Julia

```
@BI function model_binomial(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                  event, divisor, dyad_ids, design_fixed, design_random, random_group_index
                  prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                  prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                  prior_zero_prob_alpha, prior_zero_prob_beta,
                  priors_only, partial_pooling, zero_inflated)

    # Edge weights
    if partial_pooling == 0
        edge_weight = m.dist.normal(prior_edge_mu, prior_edge_sigma, shape=(num_edges,))
    else
        edge_sigma = m.dist.normal(0, prior_edge_sigma, shape=(1,), lower=0)
        edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[1], shape=(num_edges,))
    end

    # Fixed & random
    beta_fixed = 0
    if num_fixed > 0
        beta_fixed = m.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=(num_fixed,))
    end

    beta_random = 0
    if num_random > 0
        random_group_mu = m.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape=
        random_group_sigma = m.dist.normal(0, prior_random_std_sigma, shape=(num_random_grou
        beta_random = m.dist.normal(random_group_mu[random_group_index], random_group_sigma[
    end

    # Predictor
    predictor = jnp.zeros(num_rows)
    if num_edges > 0
        predictor = predictor .+ edge_weight[dyad_ids]
    end
    if num_fixed > 0
        predictor = predictor .+ jnp.dot(design_fixed, beta_fixed)
    end
    if num_random > 0
        predictor = predictor .+ jnp.dot(design_random, beta_random)
    end
```

```
    if !priors_only
        p = m.link.inv_logit(predictor)
        if zero_inflated == 0
            m.dist.binomial(total_count=divisor, probs=p, obs=event)
        else
            zero_prob = m.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=(1,))
            m.dist.zero_inflated_binomial(gate=zero_prob[1], total_count=divisor, probs=p, ob
        end
    end
end
```

---

## 2. Poisson Edge Weight Model

When working with discrete counts of interactions (e.g., number of grooming iterations, frequency of co-location), a **Poisson** approach is necessary. We include the `divisor` as an offset/exposure modifier (`exp(predictor) * divisor`).

### Python

```
@BI
def model_poisson(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                  event, divisor, dyad_ids, design_fixed, design_random, random_group_index,
                  prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                  prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                  prior_zero_prob_alpha, prior_zero_prob_beta,
                  priors_only, partial_pooling, zero_inflated):

    # Base priors structure is similar...
    if partial_pooling == 0:
        edge_weight = m.dist.normal(prior_edge_mu, prior_edge_sigma, shape=(num_edges,))
    else:
        edge_sigma = m.dist.normal(0, prior_edge_sigma, shape=(1,), lower=0)
        edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[0], shape=(num_edges,))

    beta_fixed = 0
    if num_fixed > 0:
        beta_fixed = m.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=(num_fixed,))
```

```
    beta_random = 0
    if num_random > 0:
        random_group_mu = m.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape=
        random_group_sigma = m.dist.normal(0, prior_random_std_sigma, shape=(num_random_grou
        beta_random = m.dist.normal(random_group_mu[random_group_index], random_group_sigma[

    predictor = jnp.zeros(num_rows)
    if num_edges > 0:
        predictor += edge_weight[dyad_ids]
    if num_fixed > 0:
        predictor += jnp.dot(design_fixed, beta_fixed)
    if num_random > 0:
        predictor += jnp.dot(design_random, beta_random)

    if not priors_only:
        rate = jnp.exp(predictor) * divisor
        if zero_inflated == 0:
            m.dist.poisson(rate, obs=event)
        else:
            zero_prob = m.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=(1,))
            m.dist.zero_inflated_poisson(gate=zero_prob[0], rate=rate, obs=event)
```

**R**

```
model_poisson <- function(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                  event, divisor, dyad_ids, design_fixed, design_random, random_group_index,
                  prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                  prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                  prior_zero_prob_alpha, prior_zero_prob_beta,
                  priors_only, partial_pooling, zero_inflated) {

    if (partial_pooling == 0) {
        edge_weight = bi.dist.normal(prior_edge_mu, prior_edge_sigma, shape=c(num_edges))
    } else {
        edge_sigma = bi.dist.normal(0, prior_edge_sigma, shape=c(1), lower=0)
        edge_weight = bi.dist.normal(prior_edge_mu, edge_sigma[1], shape=c(num_edges))
    }

    beta_fixed = 0
    if (num_fixed > 0) {
        beta_fixed = bi.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=c(num_fixed))
```

```
    }

    beta_random = 0
    if (num_random > 0) {
        random_group_mu = bi.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape
        random_group_sigma = bi.dist.normal(0, prior_random_std_sigma, shape=c(num_random_gr
        beta_random = bi.dist.normal(random_group_mu[random_group_index], random_group_sigma
    }

    predictor = jnp$zeros(num_rows)
    if (num_edges > 0) {
        predictor = predictor + as.vector(edge_weight[dyad_ids])
    }
    if (num_fixed > 0) {
        predictor = predictor + jnp$dot(design_fixed, beta_fixed)
    }
    if (num_random > 0) {
        predictor = predictor + jnp$dot(design_random, beta_random)
    }

    if (!priors_only) {
        rate = jnp$exp(predictor) * divisor
        if (zero_inflated == 0) {
            bi.dist.poisson(rate, obs=event)
        } else {
            zero_prob = bi.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=c(1)
            bi.dist.zero_inflated_poisson(gate=zero_prob[1], rate=rate, obs=event)
        }
    }
}
```

### Julia

```
@BI function model_poisson(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                  event, divisor, dyad_ids, design_fixed, design_random, random_group_index,
                  prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                  prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                  prior_zero_prob_alpha, prior_zero_prob_beta,
                  priors_only, partial_pooling, zero_inflated)

    if partial_pooling == 0
```

```
        edge_weight = m.dist.normal(prior_edge_mu, prior_edge_sigma, shape=(num_edges,))
    else
        edge_sigma = m.dist.normal(0, prior_edge_sigma, shape=(1,), lower=0)
        edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[1], shape=(num_edges,))
    end

    beta_fixed = 0
    if num_fixed > 0
        beta_fixed = m.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=(num_fixed,))
    end

    beta_random = 0
    if num_random > 0
        random_group_mu = m.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape=
        random_group_sigma = m.dist.normal(0, prior_random_std_sigma, shape=(num_random_grou
        beta_random = m.dist.normal(random_group_mu[random_group_index], random_group_sigma[
    end

    predictor = jnp.zeros(num_rows)
    if num_edges > 0
        predictor = predictor .+ edge_weight[dyad_ids]
    end
    if num_fixed > 0
        predictor = predictor .+ jnp.dot(design_fixed, beta_fixed)
    end
    if num_random > 0
        predictor = predictor .+ jnp.dot(design_random, beta_random)
    end

    if !priors_only
        rate = jnp.exp(predictor) .* divisor
        if zero_inflated == 0
            m.dist.poisson(rate, obs=event)
        else
            zero_prob = m.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=(1,))
            m.dist.zero_inflated_poisson(gate=zero_prob[1], rate=rate, obs=event)
        end
    end
end
```

---

## 3. Exponential Edge Weight with Sender-Receiver Effects

In continuous time-to-event interactions, the outcome (`event` representing length of time, and `event_count` representing raw occurrence) can be modeled with an Exponential-Poisson fusion. This framework often links the tendency to emit edges strongly to receiving edges through **Sender-Receiver covariances**.

### Python

```
@BI
def model_exponential(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                      event, event_count, divisor, dyad_ids, dyad_ids_receiver,
                      design_fixed, design_random, random_group_index,
                      prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                      prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                      prior_zero_prob_alpha, prior_zero_prob_beta, prior_rate_sigma,
                      priors_only, partial_pooling, zero_inflated, sender_receiver):

    rate = m.dist.normal(0, prior_rate_sigma, shape=(num_edges,), lower=0)

    # Edge weights with sender_receiver covariance array logic mapping to BI syntax
    if partial_pooling == 0:
        if sender_receiver:
            sender_receiver_cov = m.dist.normal(0, prior_edge_sigma, shape=(1,))
            cov_mat = jnp.array([[prior_edge_sigma, sender_receiver_cov[0]],
                                 [sender_receiver_cov[0], prior_edge_sigma]])
            edge_params = m.dist.multivariate_normal(jnp.zeros(2), cov_mat, shape=(num_edges
            edge_weight = edge_params[:, 0]  # Simplified logic translation
        else:
            edge_weight = m.dist.normal(prior_edge_mu, prior_edge_sigma, shape=(num_edges,))
    else:
        edge_sigma = m.dist.normal(0, prior_edge_sigma, shape=(1,), lower=0)
        if sender_receiver:
            sender_receiver_cov = m.dist.normal(0, prior_edge_sigma, shape=(1,))
            cov_mat = jnp.array([[edge_sigma[0], sender_receiver_cov[0]],
                                 [sender_receiver_cov[0], edge_sigma[0]]])
            edge_params = m.dist.multivariate_normal(jnp.zeros(2), cov_mat, shape=(num_edges
            edge_weight = edge_params[:, 0]
        else:
            edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[0], shape=(num_edges,))
```

```
    # Fixed & random effects
    beta_fixed = 0
    if num_fixed > 0:
        beta_fixed = m.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=(num_fixed,))

    beta_random = 0
    if num_random > 0:
        random_group_mu = m.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape=
        random_group_sigma = m.dist.normal(0, prior_random_std_sigma, shape=(num_random_grou
        beta_random = m.dist.normal(random_group_mu[random_group_index], random_group_sigma[

    predictor = jnp.zeros(num_rows)
    if num_edges > 0:
        predictor += edge_weight[dyad_ids]
    if num_fixed > 0:
        predictor += jnp.dot(design_fixed, beta_fixed)
    if num_random > 0:
        predictor += jnp.dot(design_random, beta_random)

    if not priors_only:
        time_rate = rate[dyad_ids] / m.link.inv_logit(predictor)
        if zero_inflated == 0:
            m.dist.exponential(time_rate, obs=event)
            m.dist.poisson(rate * divisor, obs=event_count)
        else:
            zero_prob = m.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=(1,))
            m.dist.zero_inflated_exponential(gate=zero_prob[0], rate=time_rate, obs=event)
```

**R**

```
model_exponential <- function(num_rows, num_edges, num_fixed, num_random, num_random_groups,
                      event, event_count, divisor, dyad_ids, dyad_ids_receiver,
                      design_fixed, design_random, random_group_index,
                      prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                      prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                      prior_zero_prob_alpha, prior_zero_prob_beta, prior_rate_sigma,
                      priors_only, partial_pooling, zero_inflated, sender_receiver) {

    rate = bi.dist.normal(0, prior_rate_sigma, shape=c(num_edges), lower=0)

    if (partial_pooling == 0) {
```

```
        if (sender_receiver) {
            sender_receiver_cov = bi.dist.normal(0, prior_edge_sigma, shape=c(1))
            # ... Multivariate Normal constructor logic natively mapped ...
            edge_weight = bi.dist.normal(0, prior_edge_sigma, shape=c(num_edges)) # Substitu
        } else {
            edge_weight = bi.dist.normal(prior_edge_mu, prior_edge_sigma, shape=c(num_edges)
        }
    } else {
        edge_sigma = bi.dist.normal(0, prior_edge_sigma, shape=c(1), lower=0)
        if (sender_receiver) {
            # ... Multivariate constructor ...
            edge_weight = bi.dist.normal(prior_edge_mu, edge_sigma[1], shape=c(num_edges))
        } else {
            edge_weight = bi.dist.normal(prior_edge_mu, edge_sigma[1], shape=c(num_edges))
        }
    }

    beta_fixed = 0
    if (num_fixed > 0) {
        beta_fixed = bi.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=c(num_fixed))
    }

    beta_random = 0
    if (num_random > 0) {
        random_group_mu = bi.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape
        random_group_sigma = bi.dist.normal(0, prior_random_std_sigma, shape=c(num_random_gr
        beta_random = bi.dist.normal(random_group_mu[random_group_index], random_group_sigma
    }

    predictor = jnp$zeros(num_rows)
    if (num_edges > 0) {
        predictor = predictor + as.vector(edge_weight[dyad_ids])
    }
    if (num_fixed > 0) {
        predictor = predictor + jnp$dot(design_fixed, beta_fixed)
    }
    if (num_random > 0) {
        predictor = predictor + jnp$dot(design_random, beta_random)
    }

    if (!priors_only) {
        time_rate = rate[dyad_ids] / m$link$inv_logit(predictor)
```

```
        if (zero_inflated == 0) {
            bi.dist.exponential(time_rate, obs=event)
            bi.dist.poisson(rate * divisor, obs=event_count)
        } else {
            zero_prob = bi.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=c(1))
            bi.dist.zero_inflated_exponential(gate=zero_prob[1], rate=time_rate, obs=event)
        }
    }
}
```

### Julia

```
@BI function model_exponential(num_rows, num_edges, num_fixed, num_random, num_random_groups
                      event, event_count, divisor, dyad_ids, dyad_ids_receiver,
                      design_fixed, design_random, random_group_index,
                      prior_edge_mu, prior_edge_sigma, prior_fixed_mu, prior_fixed_sigma,
                      prior_random_mean_mu, prior_random_mean_sigma, prior_random_std_sigma,
                      prior_zero_prob_alpha, prior_zero_prob_beta, prior_rate_sigma,
                      priors_only, partial_pooling, zero_inflated, sender_receiver)

    rate = m.dist.normal(0, prior_rate_sigma, shape=(num_edges,), lower=0)

    if partial_pooling == 0
        if sender_receiver
            sender_receiver_cov = m.dist.normal(0, prior_edge_sigma, shape=(1,))
            # Substitute implementation for multi-normal mapping
            edge_weight = m.dist.normal(0, prior_edge_sigma, shape=(num_edges,))
        else
            edge_weight = m.dist.normal(prior_edge_mu, prior_edge_sigma, shape=(num_edges,))
        end
    else
        edge_sigma = m.dist.normal(0, prior_edge_sigma, shape=(1,), lower=0)
        if sender_receiver
            edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[1], shape=(num_edges,))
        else
            edge_weight = m.dist.normal(prior_edge_mu, edge_sigma[1], shape=(num_edges,))
        end
    end

    beta_fixed = 0
    if num_fixed > 0
```

```
        beta_fixed = m.dist.normal(prior_fixed_mu, prior_fixed_sigma, shape=(num_fixed,))
    end

    beta_random = 0
    if num_random > 0
        random_group_mu = m.dist.normal(prior_random_mean_mu, prior_random_mean_sigma, shape=
        random_group_sigma = m.dist.normal(0, prior_random_std_sigma, shape=(num_random_grou
        beta_random = m.dist.normal(random_group_mu[random_group_index], random_group_sigma[
    end

    predictor = jnp.zeros(num_rows)
    if num_edges > 0
        predictor = predictor .+ edge_weight[dyad_ids]
    end
    if num_fixed > 0
        predictor = predictor .+ jnp.dot(design_fixed, beta_fixed)
    end
    if num_random > 0
        predictor = predictor .+ jnp.dot(design_random, beta_random)
    end

    if !priors_only
        time_rate = rate[dyad_ids] ./ m.link.inv_logit(predictor)
        if zero_inflated == 0
            m.dist.exponential(time_rate, obs=event)
            m.dist.poisson(rate .* divisor, obs=event_count)
        else
            zero_prob = m.dist.beta(prior_zero_prob_alpha, prior_zero_prob_beta, shape=(1,))
            m.dist.zero_inflated_exponential(gate=zero_prob[1], rate=time_rate, obs=event)
        end
    end
end
```